

软件体系结构

一、题型

1. 名词解释（10 分、5 个）
2. 按功能要求书写命令（10 分，5 个）
3. 简答（40 分 5-6 个）
4. 材料阅读（40 分 2 个）

二、名词解释

1. **软件危机**：指落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列严重问题的现象。
2. **软件重用**：指在两次或多次不同的软件开发过程中重复使用相同或相似软件元素的过程。
3. **CDL**：基于软件构件的描述语言，由许多以特定形式相互作用的特殊软件实体构造组成的组织或系统。
4. **MIL**：模块内联语言，是将一种或多种传统程序设计语言模块连接起来描述软件的体系结构的方法。
5. **ADL**：软件体系结构描述语言，参照传统程序设计语言和开发经验，针对软件的结构特点，重新设计和开发的描述方式，可以在指定的抽象层次上描述软件体系结构。
6. **SAAM**：软件架构分析方法，它试图通过场景来测量软件的质量，而不是泛泛的不精确的质量属性描述。
7. **ATAM**：体系结构权衡分析方法，是一种能够权衡多个相关甚至是不一致的质量需求或者目标的评估方法。
8. **ARID**：积极的中间设计审核方法，用于软件体系结构设计过程之中，针对于未完成的体系结构。
9. **技术债**：技术债是指开发人员为了加速软件开发，或是由于自身经验的缺乏，有意或无意地在应该采用最佳方案时进行了妥协，使用了短期内能加速软件开发的方案，从而在未来给自己带来的额外开发负担。
10. **技术破产**：随着软件发生持续变化，总的技术债一直增加，当累积的技术债达到无法偿还的地步，使得不得不放弃该软件产品。
11. **代码坏味道**：如果程序中的某一段代码是不稳定或者有一些潜在问题的，那么该段代码往往会包含一些明显的不太好的痕迹。
12. **设计坏味道**：软件设计过程中，由于各种原因（人员素质、技术方法、技术债等原因）也会引起坏味道，我们称之为设计坏味道
13. **软件脆弱性**：导致破坏系统安全策略的系统安全规范、系统设计、实现和内部控制等方面的弱点。

三、绪论

1. 软件体系结构思想的诞生时间：1960s。
2. 软件体系结构三要素：组件（构件）+连接件+约束。

3. **软件重用**：两次或多次重复使用相同或相似的元素，目的解决软件危机。
4. **软件危机，软件危机的表现有哪些**：软件开发进度难以预测、软件开发成本难以控制、用户对产品功能难以满足、软件产品质量无法保证、软件产品难以维护。
5. **软件体系结构在软件生命周期中的作用**：项目规划、需求分析、软件设计、软件实现、测试与评审、维护与升级。

四、SA 风格

1. 软件体系结构风格不能作为软件分类的标准。
2. **列举典型的体系结构风格**：管道过滤器风格、主程序/子程序风格、事件驱动风格、分层风格、面向对象风格、解释器风格、基于规则的系统风格、仓库风格、黑板系统风格、C2 风格、客户机/服务器风格、浏览器/服务器风格、正交架构风格、模型-视图-控制器风格、基于层次消息总线的架构风格、平台/插件风格、面向服务架构风格、云体系结构风格、网格计算体系结构风格、微服务架构、异构风格的集成。
3. **管道过滤器风格**：
 - (1) **概念**：每个功能模块都有一组输入和输出。功能模块称作过滤器（filters）；功能模块间的连接可以看作输入、输出数据流之间的通路，所以称作管道（pipes）。
 - (2) **优点**：整个系统的行为易于理解；过滤器可以重用/重组合；系统易于维护和扩展；支持一些特定的分析，如吞吐量计算和死锁检测；高效的并行处理（多主动部件）。
 - (3) **缺点**：交互式处理能力弱；数据传输开销大；错误处理较为复杂。
 - (4) **应用**：命令行、编译器、网络通信协议。
4. **主程序/子程序风格**：
 - (1) **概念**：组件为主程序和子程序，连接件为调用-返回机制，拓扑结构为层次化结构。
 - (2) **优点**：具有很高的数据访问效率，因为计算共享同一个存储区；不同的计算功能被划分在不同的模块中。
 - (3) **缺点**：对数据存储格式的变化将会影响几乎所有的模块；对处理流程的改变与系统功能的增强也很难适应，依赖于控制模块内部的调用次序；这种分解方案难以支持有效的复用。
5. **事件驱动风格**：
 - (1) **概念**：事件驱动系统的基本观点是一个系统对外部的表现可以从它对事件的处理表征出来。
 - (2) **优点**：非常适合于描述系统族，在属于同一族的任何系统中，系统的高级管理子系统的描述是完全类似的，便于重用；由于最高管理子系统牢牢的掌握着控制权，又因为各同级子系统一般不直接发生关系，因此容易实现并发处理和多任务操作；具有良好的可扩展性，设计者只需为某个对象注册一个事件处理接口就可以将该对象引入整个系统，同时并不影响其它的系统对象；定义了包含执行子系统和管理子系统的类层次结构；简化客户代码；使整个系统的设计更具有一般化。
 - (3) **缺点**：构件削弱了自身对系统计算的控制能力；存在数据交换问题；系统中各个对象的逻辑关系变得更加复杂。
 - (4) **应用**：JavaBean 系统。
6. **分层风格**：
 - (1) **概念**：每个层次由一系列组件构成，层次之间存在接口。

(2) **优点：**分层风格支持系统设计过程中的逐级抽象；基于分层风格的系统具有较好的可扩展性；分层风格支持软件复用。

(3) **缺点：**并不是所有的系统都适合用分层风格来描述的；对于抽象出来的功能具体应该放在哪个层次上是一个难题。

(4) **应用：**计算机网络的设计。

7. 面向对象风格：

(1) **概念：**面向对象模式集数据抽象、抽象数据类型、类继承为一体，使软件工程公认的模块化、信息隐藏、抽象、重用性等原则在面向对象风格下得以充分实现。

(2) **优点：**高度模块性，数据与相关操作被组织为对象，成为模块组织基本单位；封装功能，一组功能和其实现细节被封装在一个对象中，具有功能的接口被暴露出来；代码共享，对象的相对独立性可被反复重用，通过拼装形成不同软件系统；灵活性，对象在组织过程中，相互关系可以任意变化，只要接口兼容；易维护性，对象接近于人对问题和解决方案模型的思维方式，易于理解和修改。

(3) **缺点：**如果一个对象需要调用另一个对象，它就必须知道那个对象的标识（对象名或对象引用），无形之中增强了对对象之间的依赖关系。如果一个对象改变了自己的标识，就必须通知系统中所有和它有调用关系的对象，否则系统就无法正常运行。

(4) **应用：**人事档案管理系统。

8. 客户机/服务器风格（C/S 风格）：

(1) **概念：**客户机和服务器是两个相互独立的逻辑系统，为完成特定任务而形成一种协作关系。客户机(前端)：业务逻辑、与服务器通讯的接口；服务器(后端)：与客户机通讯的接口、业务逻辑、数据管理。一般的，客户机为完成特定的工作向服务器发出请求；服务器处理客户机的请求并返回结果。客户机程序和服务器程序配置在同一台计算机上时：采用消息、共享存储区和信号量等方法实现通信连接。客户机程序和服务器程序配置在分布式环境中时：通过远程调用（Remote Procedure Call, RPC）协议进行通信。

(2) **优点：**

1) **两层 C/S 架构：**客户机组件和服务器组件分别运行在不同的计算机上，有利于分布式数据的组织和处理；组件之间的位置是相互透明的；客户机程序和服务器程序可运行在不同的操作系统上，便于实现异构环境和多种不同开发技术的融合；软件环境和硬件环境的配置具有极大的灵活性，易于系统功能的扩展；将大规模的业务逻辑分布到多个通过网络连接的低成本的计算机上，降低了系统的整体开销。

2) **三层 C/S 架构相比于两层 C/S 架构：**合理地划分三层结构的功能，可以使系统的逻辑结构更加清晰，提高软件的可维护性和可扩充性；在实现三层 C/S 架构时，可以更有效地选择运行平台和硬件环境，从而使每一层都具有清晰的逻辑结构、良好的负荷处理能力和较好的开放性；在 C/S 架构中，可以分别选择合适的编程语言并行开发；系统具有较高的安全性。

(3) **缺点：****两层 C/S 架构：**开发成本较高（客户机的软硬件要求高）；客户机程序的设计复杂度大，客户机负荷重；信息内容和形式单一；C/S 架构升级需要开发人员到现场更新客户机程序，对运行环境进行重新配置，增加了维护费用；两层 C/S 结构采用了单一的服务器，同时以局域网为中心，难以扩展到 Intranet 和 Internet；数据安全性不高。

(4) **使用三层 C/S 架构时需注意以下两个问题：**如果各层之间的通信效率不高，即使每一层的硬件配置都很高，系统的整体性能也不会太高；必须慎重考虑三层之间的通信方法、通信频率和传输数据量，这和提高各层的独立性一样也是实现三层 C/S 架构的关键性问题。

(5) 应用：微信、QQ、支付宝。

9. 浏览器/服务器风格（B/S 风格）：

(1) 概念：浏览器/服务器风格是三层 C/S 风格的一种实现方式。主要包括浏览器，Web 服务器和数据库服务器。与三层 C/S 结构的解决方案相比，B/S 架构在客户机上采用了 WWW 浏览器，将 Web 服务器作为应用服务器。B/S 架构核心是 Web 服务器，数据请求、网页生成、数据库访问和应用程序执行全部由 Web 服务器来完成。在 B/S 架构中，系统安装、修改和维护全在服务器端解决，客户端无任何业务逻辑。

(2) 优点：客户端只需要安装浏览器，操作简单，能够发布动态信息和静态信息；运用 HTTP 标准协议和统一客户端软件，能够实现跨平台通信；开发成本较低，只需维护 Web 服务器程序和中心数据库。客户端升级可以通过升级浏览器来实现。

(3) 缺点：个性化程度较低，所有客户端程序的功能都是一样的；客户端数据处理能力比较差，加重 Web 服务器的工作负担，影响系统整体性能；数据提交一般以页面为单位，动态交互性不强，不利于在线事务处理；可扩展性较差，系统安全性难以保障；应用系统查询中心数据库，其速度远低于 C/S 架构。

10. 模型-视图-控制器风格（MVC 风格）：

(1) 概念：MVC 结构主要包括模型、视图和控制器三部分。模型(Model, M)：模型是应用程序核心，封装了问题的核心数据、逻辑关系和计算功能，提供处理问题的操作过程；视图(View, V)：视图是模型的表示，提供了交互界面，为用户显示模型信息；控制器(Controller, C)：控制器负责处理用户与系统之间的交互，为用户提供操作接口。

(2) 优点：多个视图与一个模型相对应；具有良好的移植性；系统被分割为三个独立的部分，当功能发生变化时，改变其中的一个部分就能满足要求。

(3) 缺点：增加系统设计和运行复杂性；视图与控制器连接过于紧密，妨碍二者的独立重用；视图访问模型的效率较低；频繁访问未变化的数据，也将降低系统的性能。

(4) 应用：Struts、Webwork、Spring MVC。

五、SA 描述

1. 软件体系结构描述的方式：非标准图形符号（UML）；模块内联语言（MIL）；基于软构件的描述语言（CDL）；软件体系结构描述语言（ADL）。

2. 非标准图形符号（UML）的优缺点：

(1) 优点：直观形象；简单易用。

(2) 缺点：术语和表达语义上存在一些不规范和不精确，使得以矩形为基础的传统图形表达方法在不同系统和不同文档之间存在着许多不一致甚至矛盾。

3. 模块内联语言（MIL）的优缺点：

(1) 优点：严格的语义基础，支持对较大软件单元进行：定义/使用(Definition/Use)、接口定义(Interface Definition)和导入/导出(Import/Export)等操作；一般来讲，MIL 与实际的实现语言无关，只关注构件的对外表现协议以及构件之间的通讯关系。

(2) 缺点：这些语言处理和描述的软件设计开发层次过于依赖程序设计语言，限制了它们处理和描述比程序设计语言元素更为抽象的高层次软件构架元素的能力。

4. 基于软构件的描述语言（CDL）的优缺点：

(1) 优点：可以描述组件安全特性及其特性；能使用时间运算符（如 next 和 Lead-to）描述进度属性。

(2) 缺点：应用范围有局限性，不适合一般的体系结构描述。

5. 软件体系结构描述语言（ADL）的优缺点：

- (1) **优点：**语义精确；良好的整体性；良好的抽象性。
- (2) **缺点：**没有普遍一致的意见：ADL 应表现什么，特别是架构的行为；目前使用的表现，分析困难且无商业工具支持；大多数 ADL 倾向于垂直优化特定的分析架构的共同概念。

6. 4+1 视图：

- (1) **4 指的内容：**逻辑视图（给用户看）、开发视图（给程序员看）、过程视图（给系统集成人员看）、物理视图（给部署人员看）。
- (2) **1 指的内容：**场景视图（给最终使用用户或软件分析人员看）。
- (3) **系统内用户关心的视图：**最终用户关心系统功能，侧重逻辑视图；程序员关心系统开发、配置等问题，侧重开发视图；系统集成人员关心系统性能、可伸缩性、吞吐率等问题，侧重进程(处理)视图；系统工程师关心系统的发布、安装、拓扑结构等问题，侧重部署(物理)视图。分析人员和测试人员关心系统的行为，侧重于用例(场景)视图；

7. 主要的形式化 ADL 语言有那些，适用于哪些应用场景？

编号	典型的ADLs	应用场景
1	Aesop	特定领域的软件系统
2	C2 SADL	用户界面密集系统
3	MetaH	实时、容错、安全等系统
4	UniCon	大量组件和连接件的管理
5	Rapide	基于事件的、复杂、并发、分布式系统
6	Wright	复杂交互
7	Darwin	基于消息传递的分布式系统
8	XYZ/ADL	架构逐层细化、分析、验证和自动代码的生成
9	ACME	架构在不同ADL之间变换、共享
10	XADL 2.0	实时系统、产品线架构
11	XBA	不同ADL的模型共享与协作开发
12	ABC/ADL	架构建模元素定义

六、SA 评估

1. 质量属性

- (1) **可修改性：**度量软件系统变化的成本。由变化范围、变化需要的工作量以及变化带来的商业成本进行度量的。变化包括功能扩展，容量扩展（如增加并发访问数），特性删减，数据结构更新（如对数据库的模式增加一个字段），通信协议调整，平台移植等。很多研究关注后部署阶段的修改，通常有代码级，生成级（比方说使用另外的生成选项）和配置级。可维护性、可扩展性、结构重组、可移植性。
- (2) **可用性：**指软件在发生错误、异常或者失败时候是如何反应的。一般来讲，软件的可用性可以用正常工作的概率来度量。大体上公式如下：

$$\text{可用性} = \frac{\text{平均工作时间}}{\text{平均工作时间} + \text{平均修复时间}}$$

- (3) **性能：**表征软件系统的响应速度或者由响应速度决定的其他度量。在军事、控制系统和商务信息系统中这个属性至关重要。例如：“系统必须在特定事件发生后三秒内给出答案”、“服务器必须能每秒处理 1000 个请求”。性能和可修改性，安全性有竞

争的关系，这是需要做很多权衡的地方。有些人也把性能叫做效率。

(4) 可测试性：表明软件系统在多大程度上容易被测试检查出缺陷。好的体系结构应该考虑到测试的需要。由于对已经开发完的系统进行测试代价很高昂，如果能在体系结构级别考虑到测试（从而使测试容易进行）就会有相当的回报。对可测试性的常见的度量是测试的效果和效率，比如“三天之内对模块 A 的 65% 的执行路径进行了测试”。

(5) 易用性

(6) 安全性

(7) 兼容性

(8) 可移植性

(9) 清晰性

(10) 健壮性

2. 评估方法的分类：

(1) 基于调查问卷和检查表的评估方式：这一评估方式比较灵活自由，可评估多种质量属性，也可以在软件体系结构设计的多个阶段进行。但由于评估的结果很大程度上来自评估人员的主观推断，因此不同的评估人员可能会产生不同甚至截然相反的结果。

(2) 基于场景的评估方法：将对系统质量需求转换为一系列风险承担者与系统的交互活动，分析软件体系结构对这一具体活动的支持程度。在评估过程中将考虑所有与系统相关人员(包括风险承担者)对质量的需求，从而确定应用领域功能与软件体系结构之间的映射。在此基础上设计用于体现待评估质量属性的场景以及软件体系结构对场景的支持程度。

(3) 基于度量和预测的评估方法（客观的）：度量是指为软件产品的某一属性所赋予的数值，如代码行、方法调度层数、构建个数等。类似的软件体系结构度量应该也能够作为评判质量的重要依据。建立质量属性和度量之间的映射原则：从软件体系结构文档中推导出度量信息；根据映射原理分析出系统的某些质量属性。

(4) 基于特定软件体系结构描述语言的评估方法：此类方法依赖特定的软件体系结构描述语言，与形式化或半形式化的软件结构描述语言的定义和理论基础相关。常见的体系结构描述语言有 xADL 和 UML；特定领域软件体系结构分析方法(DSSA)采用 Meta-H 描述软件体系结构模型，在基于 UML 逆向工程的软件体系结构分析方法中，利用 UML 定义特定领域的软件体系结构概念和约束。

3. 名词解释：

(1) SAAM：SAAM (Software Architecture Analysis Method, 基于场景的构架分析方法，或者叫软件构架分析方法)是最早精心设计并形成文档的分析方法。它出现于 1993 年，发表于 1994 年 (Kazman, 1994)。后来 (Kazman, 1996b) 对其进行改进，(Bass, 1998) 对其进行了几个深入的案例研究。SAAM 是一种直观的方法，试图通过场景来测量软件的质量，而不是泛泛的不精确的质量属性描述。SAAM 也比较简单，仅仅考虑场景和体系结构的关系，也不涉及太多的步骤和独特的技术。于是，它成为体系结构评估初学者的理想入门方法。

(2) ATAM：ATAM (Architecture Tradeoff Analysis Method, 体系结构权衡分析方法)，该方法可以看作 SAAM 的增强版。从名字即可看出，ATAM 方法除了能暴露被评估体系结构的潜在缺陷和风险外，还能使人们更好的理解和权衡多个相关的，或者甚至是不一致的质量需求或目标。ATAM 的基础来自三个领域：体系结构风格，质量属性分析组（包含丰富的质量属性和体系结构对应关系的库）和 SAAM。

4. 几种评估方法的比较，有什么不同：

评估方式	调查问卷或检查表		场景	度量
通用性	调查问卷	检查表	特定系统	通用或特定领域
	通用	特定领域		
评估者对体系结构的了解程度	粗略了解	无限制	中等了解	精确了解
实施阶段	早	中	中	中
客观性	主观	主观	较主观	较客观

七、SA 集成环境

1. 集成开发环境的作用：集成开发环境使开发者摆脱了繁杂的语法，语义，标识符号和公式的困扰，可以集中精力设计系统的结构；对于规模庞大，结构复杂的软件系统，资源的有效管理和利用极其重要；集成开发环境把开发过程中所需的各项功能集合在一起，实现了体系结构分析、设计、建模、验证的自动化，这是形式化方法无法取代的。

(1) 辅助体系结构建模：建立体系结构模型是体系结构集成开发环境最重要的功能之一。集成开发环境的出现增加了软件体系结构描述方法的多样性，摒弃了描述能力低的非形式化方法，摆脱了拥有繁杂语法，语义规则的形式化方法。开发者只需经过简单的操作就可以完成以前需耗费大量时间和精力的工作。集成开发环境提供了一套支持自动建模的机制完成体系结构模型分析、设计、建立、验证等过程。用户根据不同的实际需求，应用环境和体系结构等因素选择不同的开发工具。

(2) 支持层次结构的描述：随着软件系统规模越来越大、越来越复杂，简单体系结构风格无法表达结构复杂的系统。这时就需要层次结构的支持，因此开发工具也需要提供层次机制。

(3) 提供自动验证机制：几乎所有的体系结构集成开发环境都提供了体系结构验证的功能。体系结构描述语言解析器和编译器是集成开发环境中必不可少的模块。除此以外，不同的集成开发环境根据不同的要求会支持特定的检验机制。集成开发环境的测试方式可分为主动型和被动型两种。(Medvidovic, 2000) 主动型是指在错误出现之前采取预防措施，是保证系统不出现错误状态的动态策略。它根据系统当前的状态选择恰当的设计决策保证系统正常运行。被动型是指允许错误暂时存在，但最终要保证系统的正确性。

(4) 提供图形和文本操作环境：体系结构集成开发环境是开发者研究体系结构的可视化工具和展示平台，它具有友好的图形用户界面和便捷的操作环境。

(5) 支持多视图：多视图体现了关注点分离的思想，把体系结构描述语言和多视图结合起来描述系统的体系结构，能使系统更易于理解，方便系统相关人员之间相互交流，还有利于系统的一致性检测以及系统质量属性的评估。例如 Darwin 的分层系统视图，ArchStudio 的文件管理视图等。

2. 体系结构 IDE 原型的组成部分：

(1) 用户界面层：用户界面层是用户和系统交互的唯一渠道，用户需要的操作都被集成到这一层。这些操作可以通过编辑器和视图来实现。

(2) 模型层：模型层是系统的核心层，系统的大部分功能都在这一层定义和实现，它主要的任务是辅助体系结构集成开发环境建立体系结构模型。

(3) **基础层**：这一层是系统的基本保障，涵盖了系统运行所需的软硬件支撑环境，它还对系统运行时所用的资源进行管理和调度。通常，普通的简单配置就可以满足系统运行需求，但是有的体系结构集成开发环境需要更多的支持环境。

3. 主流的体系结构 IDE 有哪些：

(1) **ACMEStudio (重点)**：AcmeStudio 是卡内基梅隆大学开发的一个体系结构设计环境。它提供了一个图形界面，允许软件工程师创建各种风格的体系结构设计，并操作和分析这些设计。AcmeStudio 中使用的术语与 Acme 体系结构描述语言相同，这是一种用于捕获体系结构的可扩展文本符号。

(2) **SysADL Studio**：SysADL Studio 是 SysADL 体系结构描述语言的集成开发环境。它为基于 SysADL 的软件体系结构设计和建模提供了一个工程环境和工作台。SysADL Studio 是 Eclipse 的一个插件，使用 EMF, Xtext 和 Sirius 开发，分别用于创建图形建模工作台和开发编程语言文本编辑和领域特定语言。它们也是 Eclipse 插件，Eclipse 是一个著名的开源 Java 集成开发环境。SysADL Studio 是支持多视图建模、交叉视图检查、执行和验证的集成开发环境。SysADL Studio 支持 SysADL 语言模型的多视角建模，从而允许架构师在每个视角中编辑图，定义图元之间的关系并执行它们。提供的工具分别是图编辑器，分配透视图，文本编辑器和执行透视图。还有一种验证机制可以验证体系结构是否满足一组语言约束。

(3) **ArchStudio5**：ArchStudio5 是美国加利福尼亚大学欧文分校的软件研究实验室开发的面向体系结构的基于 xADL3.0 的开源集成开发环境。ArchStudio5 在前一版的基础上添加了新的特性和功能，在可扩展性，系统实施和工程性上有新的发展。ArchStudio5 的作用主要体现在基本功能和扩展功能两方面。它不但实现了建模，可视化，检测和系统实施等基本功能还良好地支持这些功能的扩展。

八、SA 技术债与坏味道

1. **技术债**：技术债是指开发人员为了加速软件开发，或是由于自身经验的缺乏，有意或无意地在应该采用最佳方案时进行了妥协，使用了短期内能加速软件开发的方案，从而在未来给自己带来的额外开发负担。

2. **技术破产**：只要软件发生持续变化，总的技术债就会一直增加，终于有一天会使累积的技术债达到无法偿还的地步，使得不得不放弃该软件产品，这种情况被称为“技术破产”。

3. 技术债分类：

(1) **代码债**：开发时没有守代码规范导致的债务，来源包括静态分析工具的违规行为和不一致的编码风格。

(2) **设计债**：在设计时未采用最优解决方案导致的债务，来源包括设计臭味和违背设计规则的行为。架构技术债也是设计债的一种。

(3) **测试债**：测试环节产生的债务，一般由测试的缺乏、测试覆盖面不充分以及不恰当的测试设计导致。

(4) **文档债**：由技术文档问题产生的债务，包括缺少重要技术文档、较差的文档和未及时修改更新的文档。

4. 产生技术债的根本原因：

(1) **进度压力**：当软件开发接近项目截止日期时，就会努力尽快完成工作，容易仓促行事；

(2) **软件设计师缺乏足够的经验和技巧**：如果软件设计师缺乏对软件设计基础和原则

的理解，项目就容易出现各种问题：

(3) **不注重设计原则的应用**：开发者如果缺乏根据通用原则设计和编写代码的意识或经验，很容易代码难以扩展或修改，尤其是在团队开发或需要不断更新的项目中；

(4) **缺乏对设计坏味和重构的意识**：很多开发者并不注意设计坏味，而这随着时间的蔓延扩大会导致较差的软件结构质量，产生技术债。

(5) **开发中有意采用非最优的选择**：在某些项目中，有时会有意地做出一些会产生技术债的决定，从而达到某些更重要的目的，同时充分认识到技术债的存在。

5. 代码债都有哪些表现？怎样去发现它？

类型	影响
不能均匀分布的复杂性	较高的圈复杂度需要更多的测试才能覆盖到全路径，导致潜在的功能质量风险。
重复代码	重复代码是最严重的问题，会导致潜在缺陷。另外，重复也会带来维护成本的增加。
不合适的注释	代码的注释没有明确标准。缺少关键环节的注释或者是注释难以理解，都会导致代码的可读性变差。
违反代码规范	影像团队基于共同的规范进行写作，会增加潜在的风险。
缺乏单元测试	单元测试不足会影响团队对代码的信心，增加重构成本，通过测试覆盖率对其进行度量。
缺陷和潜在的缺陷	缺陷和潜在缺陷是最直接影响代码质量的要素，要尽可能地发现并修复。
设计和系统架构	设计和系统架构受限于当时的资源条件，可能无法满足后续产品的发展需求，所以需要持续演进。

6. 面向对象程序中可能出现的代码坏味道：

(1) **应用级的坏味道**：当软件开发者接近项目截止日期时，就会努力尽快完成工作，容易仓促行事。

(2) **类级的坏味道**：如果软件设计师缺乏对软件设计基础和原则的理解，项目就容易出现各种问题。

(3) **方法级的坏味道**：开发者如果缺乏根据通用原则设计和编写代码的意识或经验，很容易代码难以扩展或修改，尤其是在团队开发或需要不断更新的项目中。

7. 应用级坏味道和方法级坏味道表现：

(1) 应用级的坏味道：

- 1) **重复代码 (Duplicated code)**：在一个以上的地方出现了重复的代码片段。重复的代码结构使程序变得冗长，需要重构。
- 2) **人为复杂性 (Contrived complexity)**：在进行简单设计就充分的情况下，强制使用了过于复杂的设计模式。
- 3) **散弹式修改 (Shotgun surgery)**：如果每遇到某种变化，你都必须在许多不同的类中做出许多小修改。

(2) 类级的坏味道：

- 1) **过大的类**：如果想利用单个类做太多事，类中就会出现太多实例变量。
- 2) **依恋情结的类**：一个类的方法过多地使用了另一个的方法。
- 3) **过度亲密类**：一个类存在与另一个类的实现细节依赖。

- 4) **拒绝的馈赠**：子类仅仅使用父类中的部分方法和属性，其他来自父类的馈赠成为了累赘。
- 5) **冗赘类 (Lazy class)**：一个做的事情太少的类。
- 6) **数据泥团 (Data clump)**：当一群变量一起在程序的不同部分传播时会发生数据泥团现象。即：常常可以在很多地方看到相同的若干项数据。
- 7) **不完美的程序库类 (Incomplete Library Class)**：尽管许多编程技术都建立在 library classes 的基础上，但是 Library classes 构筑者没有未卜先知的能力，不可能考虑所有可能的情况。
- 8) **纯稚的数据类 (Data Class)**：所谓 Data Class 是指：它们拥有一些字段，以及用于访问这些字段的方法，除此之外一无长物。这样的 classes 只是一种“不会说话的数据容器”，它们几乎一定被其它 classes 过份细琐地操控着。

8. 典型的设计坏味道：

- (1) 连接件嫉妒 (Connector Envy)
- (2) 过度分散的功能 (Scattered Functionality)
- (3) 模糊接口 (Ambiguous Interface)
- (4) 无关的相邻连接件 (Extraneous Adjacent Connector)

九、SA 脆弱性

1. **软件脆弱性**：软件脆弱性是导致破坏系统安全策略的系统安全规范、系统设计、实现和内部控制等方面的弱点。在软件开发过程中，软件脆弱性包含软件基础模型的脆弱性、软件架构设计的脆弱性、软件模块设计的脆弱性、软件接口设计的脆弱性、软件界面设计的脆弱性、数据库设计的脆弱性、架构模式和设计模式的脆弱性以及实现的脆弱性等等。

2. 软件脆弱性特点：

- (1) 脆弱性是软件系统中隐藏的一个弱点，本身不引起危害，但被利用后会产生严重的安全后果。
- (2) 在软件开发过程中，自觉或不自觉引入的逻辑错误是大多数脆弱性的根本来源。
- (3) 与具体系统环境密切相关，系统环境的任何差异都有可能引起不同的脆弱性问题。
- (4) 旧的脆弱性得到修补或纠正的同时可能会引入新的脆弱性。

3. **软件架构脆弱性**：软件架构脆弱性属于软件设计脆弱性或软件结构脆弱性的一种。软件架构脆弱性是更高层次的软件结构脆弱性，也是更加重要的软件脆弱性问题。软件（系统）架构设计存在一些明显的或者隐含的缺陷，攻击者可以利用这些缺陷攻击系统，或者当受到某个或某些外部刺激时，系统发生性能下降、稳定性下降、可靠性下降、安全性下降等等。如果软件架构具备这类缺陷，我们认为该软件架构是脆弱的，也就是软件架构脆弱性。软件架构脆弱性通常与软件架构的风格 (style) 和模式 (pattern) 有关，不同风格和模式的软件架构，脆弱性体现和特点有很大不同。

4. 特定风格的脆弱性：

- (1) **管道/过滤器架构**：每个过滤器的输入输出是独立的，一个过滤器的输出做为另外一个过滤器的输入，耦合度低；若攻击者知道了某个过滤器的输入输出，有可能得出过滤器的功能，从而对系统的安全性造成威胁；由于一个过滤器的输出作为另外一个过滤器的输入，若一个过滤器发生错误，使得整个错误在系统中放大，从而威胁系统的稳定性。

(2) 事件驱动架构：组件削弱了自身对系统的控制能力。一个组件触及事件时，并不能确定响应该事件的其他组件及各组件的执行顺序；不能很好地解决数据交换问题；使系统中各组件的逻辑关系变得更加复杂；事件驱动容易进入死循环，这是编程逻辑决定的；虽然有机会实现有效利用 cpu，但也存在高并发事件处理的可能造成系统响应问题，且易导致系统数据不正确、丢失数据现象；因为可响应的流程基本都是固定的，如果操作不当，容易引发安全问题。

(3) 黑板模式：不能确保期望结果。因为求解整个问题是通过多个知识源来进行的，而知识源的选择是通过控制机构中调度程序来选择的，如果知识源选择的不够恰当，可能会导致错误或者有误差的结果；复杂，效率低下。一个问题的求解可能需要多个知识源，并且需要排队，并且需要不停的修改黑板的状态，效率比较低；不支持并行。首先控制机构中需要将知识源放入调度队列中排队，其次多个知识源共享黑板需要同步，因为它们是通过黑板进行通讯的，每个状态的改变可能会影响下一个状态的改变。

(4) 分层架构：一旦某个底层发生错误，那么整个程序将会无法正常运行，产生一些数据溢出、空指针、空对象的安全性问题，也有可能得出错误的结果；将系统隔离为多个相对独立的层，要求在层与层之间引入通信机制。在使用面向对象方法设计的系统中，通常会存在大量细粒度的对象，以及它们之间的大量消息交互—对象成员方法的调用。本来直来直去的操作，现在要层层传递，势必造成性能的下降。

(5) 客户机/服务器结构 (C/S 结构)：客户端需要安装专用的客户端软件，系统软件升级时，每一台客户机需要重新安装，其维护和升级成本非常高；高昂的维护成本且投资大；传统的 C/S 结构的软件需要针对不同的操作系统系统开发不同版本的软件，由于产品的更新换代十分快，代价高和低效率已经不适应工作需要。

(6) 浏览器/服务器结构 (B/S 结构)：相对客户端来说，页面通用化，不突出个性。而且页面需要不断地动态刷新，尤其是当用户增多，网速慢等情况，服务器的压力将会增大，加载时间会变长。由于不需要安装客户端，客户端易扩展，B/S 将面对大量的不可知用户；相对服务器来说，当用户增多时，服务器响应速度慢。功能虽然多样化，但是不能专门化，不能实现复杂的功能。服务器承担着重要的责任，数据负荷较重。一旦发生服务器“崩溃”等问题，后果不堪设想。

(7) MVC 模式：由于没有明确的定义，完全理解 MVC 并不容易；MVC 由于将应用分为三层，意味着代码文件增多；不适合小型，中等规模的应用程序；增加系统结构和实现的复杂性；视图对模型数据的低效率访问；一般高级的界面工具或构造器不支持模式。改造这些工具以适应 MVC 需要和建立分离的部件的代价很高。

十、Hadoop 代码

1. 配置 namenode 文件格式：

```
hdfs namenode -format
```

2. 创建文件夹：

```
hadoop fs -mkdir /users 或 hdfs dfs -mkdir /users
```

3. 删除文件夹：

```
hadoop fs -rm /users/1.txt 或 hdfs dfs -rm /users/1.txt
```

4. 显示：

```
hadoop fs -cat /users/1.txt 或 hdfs dfs -cat /users/1.txt
```

5. 运行：

```
hadoop jar D:/XXX.jar wordcount /user/output
```

6. 将 D:/1.txt 传到/user 里:

```
hadoop fs -put D:/1.txt /user
```